

Java PetStore Microservices

Repo summary based on README, Maven modules, service configs, gateway routes, frontend code, and local Docker and Kubernetes assets

WHAT IT IS

An educational PetStore app refactored into Spring Boot microservices with an Angular frontend, gateway, messaging, auth, and observability support.

WHO IT'S FOR

Primary persona: Java and full stack developers learning or demonstrating how an e-commerce style application can be split into Spring Boot microservices.

WHAT IT DOES

- Shows products and lets users add new products through the Angular UI and product API.
- Accepts orders through the gateway and order service.
- Checks stock availability through inventory before saving an order.
- Publishes order-placed events to Kafka after successful orders.
- Consumes order events in notification-service and sends email confirmations.
- Exposes aggregated Swagger docs through the API gateway.
- Includes AI assistant chat endpoints in product-service via LangChain4j and OpenAI config.

HOW IT WORKS

Flow

Angular frontend calls the API gateway at localhost:9000 for product and order requests. Gateway routes /api/product, /api/order, and /api/inventory to backing services and applies circuit breaker fallbacks.

Product-service uses MongoDB. Order-service and inventory-service use MySQL with Flyway. Order-service calls inventory synchronously, saves the order, then publishes a Kafka Avro event.

Frontend Notification-service listens API Gateway sends email. Keycloak and OIDC config support auth. Grafana, Prometheus, Loki, and Tempo support observability.

Angular Bootstrap UI with Spring Cloud Gateway aggregation.

Product

REST plus MongoDB plus assistant chat endpoint.

Order

REST plus MySQL plus inventory client plus Kafka producer.

Inventory

REST plus MySQL stock checks.

Notification

Kafka consumer plus mail sender.

HOW TO RUN

1. Start local infrastructure from repo root: docker compose up -d.
2. Start the Angular app from frontend/: npm install, then npm start, and open http://localhost:4200.
3. Start backend services: explicit repo-wide startup command not found; repo evidence shows Spring Boot apps in api-gateway, product-service, order-service, inventory-service, and notification-service.
4. Optional for assistant chat: set OPENAI_API_KEY for product-service. Exact assistant usage docs: Not found in repo.

NOT FOUND IN REPO

- A single canonical backend startup command or documented local boot order.
- End-user deployment guidance or a text-based production architecture diagram.